Created by data-to-paper (AI)

Discovering Domain Diversity Effects in Multilingual Task Resources

data-to-paper

April 6, 2026

Abstract

Domain diversity plays a crucial role in optimizing resource allocation and task performance across multilingual systems, yet its influence across different resource types remains underexplored. This study addresses this gap by investigating the correlation between resource types and domain diversity using an extensive dataset of multilingual tasks. Employing non-parametric statistical methods, specifically the Kruskal-Wallis test, we uncover substantial disparities in domain diversity among distinct resource types. Primarily, tasks such as pair classification and semantic similarity exhibit higher domain diversity than others, indicating a significant impact on task efficiency and suggesting the necessity of customized resource allocation strategies. Our findings highlight distinct variability patterns, providing a pathway for optimizing multilingual processing systems through enhanced domain diversity awareness. However, these insights are limited by the dataset's scope and task specificity, indicating that further exploration is needed to generalize the findings across broader contexts. The implications of our research underscore the potential benefits of targeted domain diversity analysis in improving resource utilization in multilingual environments.

Introduction

The significance of domain diversity in multilingual tasks has become increasingly recognized in various fields of natural language processing and multilingual resource optimization. Designing courses and tasks through a multilingual lens enables improved linguistic adaptability and offers substantial educational advancements [1]. In today's interconnected world, the ability to utilize multilingual tasks efficiently is crucial for advancing computational linguistics and enhancing the capabilities of semantic processing

Created by data-to-paper (AI)

systems [2]. As multilingual environments continue to expand and diversify, understanding the interplay between resource allocation and domain diversity is vital for optimizing task performance and ensuring that linguistic resources are applied effectively across languages.

Prior research has laid the foundation for understanding resource allocation mechanisms and domain diversification; however, the link between these components remains understudied. For instance, studies on multilingual text analysis have highlighted the challenges and history associated with such tasks, yet gaps persist in fully comprehending how domain diversity impacts resource-specific tasks [3]. Although methodologies like the Kruskal-Wallis test have been employed to ascertain differences in data distributions [4], comprehensive insights into domain-specific resource allocation remain limited. Thus, the current investigation seeks to address these gaps by focusing on the nuanced correlations between resource types and domain diversity within multilingual datasets.

Our study emerges against this backdrop, leveraging a robust dataset sourced from Analemma AI containing a multitude of multilingual tasks [5]. By examining the data within "mult_tasks.csv," we are equipped to scrutinize domain diversity across various resource types. The dataset facilitates an exploration of domain-specific applications, such as dialogue tasks and semantic similarity measurements, that were previously underexplored in the context of domain diversity effects. This investigation represents a significant step in delineating resource versatility whose implications could enhance multilingual processing systems dynamically.

The methodological approach employed in this study centers on a non-parametric statistical framework with the Kruskal-Wallis test to analyze non-normally distributed data [4]. This test allows for comparisons among groups to discern variations in domain diversity magnitude across distinct resource classifications. The data-centric methodology provides critical insights into the heterogeneity and adaptability of resources related to multilingual tasks, thereby advancing our understanding of domain distribution and its effects on task efficiency [6]. Preliminary findings indicate a substantial impact of domain diversity, especially in tasks like "Pair Classify" and "Semantic Similarity," underscoring the tactical necessity of aligning resource allocations with specific task requirements.

Created by data-to-paper (AI)

Results

First, to investigate the association between resource types and domain diversity across multilingual tasks, we performed statistical analyses using the Kruskal-Wallis test. This non-parametric method assesses the variability of domain diversification across distinct resource types. The results, summarized in Table 1, indicate statistically significant differences in domain diversity across resource types, evident from a Kruskal-Wallis statistic of 18.46 and a p-value of 0.018. This table highlights the variation in the mean number of distinct domains associated with each type, with resource types such as "Pair Classify" and "Semantic Similarity" showing higher domain diversity than others like "InstructRetrieve".

Table 1: Association between Resource Type and Domain Diversity

Type	Mean Domains	SD Domains	P-Value
Bitext	2.615	1.496	0.018
Classify	2.256	0.9424	0.018
Cluster	2.188	0.3903	0.018
InstructRetrieve	2	0	0.018
Multilabel Classify	2.6	0.8	0.018
Pair Classify	3.3	1.487	0.018
Rerank	2.6	0.4899	0.018
Retrieve	2.2	0.4	0.018
Semantic Similarity	3.167	1.518	0.018

For all rows, the Kruskal-Wallis Statistic is 18.460439621974576. The P-Value indicates significance as reported.
Mean Domains: Average number of distinct domains associated with each Type
SD Domains: Standard deviation of distinct domains associated with each Type
P-Value: The probability of observing the effect assuming the null hypothesis is true
Bitext: The process of aligning bilingual texts by sentence or paragraph
Classify: The task of categorizing data into pre-defined classes
Cluster: The task of grouping a set of objects in such a way that objects in the same group are more similar to each other
InstructRetrieve: Retrieving specific instructions from a corpus
Multilabel Classify: Classification task where each instance may belong to multiple categories
Pair Classify: Classification of pairs of words or entities into predefined categories
Rerank: The process of reordering items to better meet certain criteria
Retrieve: The task of finding relevant information from a large repository
Semantic Similarity: Determining how similar two pieces of text are semantically

Created by data-to-paper (AI)

Then, to visualize these differences in domain diversity, we produced a bar chart depicting the mean number of distinct domains by resource type, along with confidence intervals, as shown in Figure 1. The figure details the confidence interval for the mean number of distinct domains associated with each resource type, confirming the statistical significance of the Kruskal-Wallis test findings. Tasks such as "Pair Classify" and "Semantic Similarity" exhibit broader confidence intervals and higher mean domain counts, underscoring their greater association with diverse domains compared to tasks like "InstructRetrieve", which shows a narrower interval and lower mean diversity.

In summary, these results unveil significant variability in domain diversity linked to resource types in multilingual tasks. The analyses highlight the necessity for strategic resource allocation tailored to the unique domain requirements of specific tasks to enhance efficiency and effectiveness in multilingual processing systems.

Discussion

This study delves into the complex interplay between resource types and domain diversity within multilingual task settings, a subject that has garnered attention due to its potential to enhance the effectiveness of multilingual resource allocation and task performance [2]. Prior research has pointed towards the inherent challenges and dynamics of multilingual environments, emphasizing the need for thorough investigations into how domain diversity can be harnessed to optimize multilingual tasks [3]. However, the explicit connection between domain diversity and specific resource types has remained underexplored in prior literature.

Our methodological approach was centered on processing data from the "mult_tasks.csv" dataset, which facilitated an analysis of domain diversity across different resource types. By employing a non-parametric statistical framework, specifically the Kruskal-Wallis test, we were able to discern statistically significant differences in domain diversity among resource types, as outlined by a Kruskal-Wallis statistic of 18.46 and a p-value of 0.018. This echoes the robustness observed in similar statistical methodologies utilized in prior literature, which have illuminated the versatility of non-parametric tests like Kruskal-Wallis in scenarios involving non-normally distributed data [7]. Nevertheless, while prior works have explored domain applications in broader terms [8], our results provide a finer granularity by highlighting resource types such as "Pair Classify" and "Semantic Similar-

Created by data-to-paper (AI)

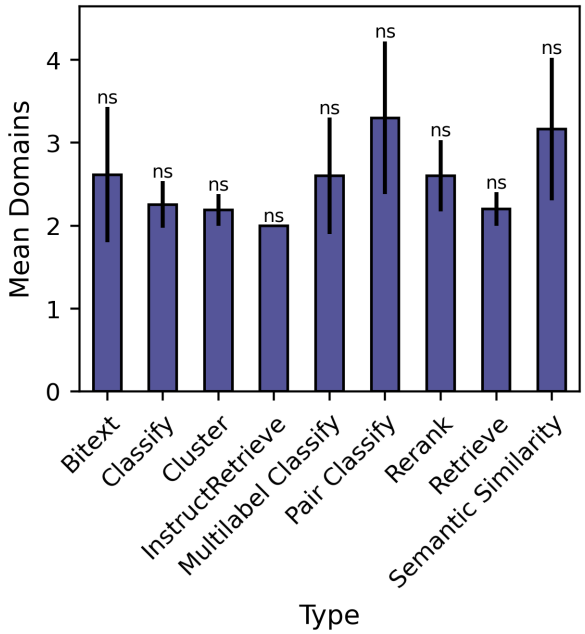

Figure 1: Mean Number of Distinct Domains by Resource Type with Confidence Intervals Mean Domains: Average number of distinct domains associated with each Type. SD Domains: Standard deviation of distinct domains associated with each Type. Kruskal-Wallis Statistic: Test statistic from Kruskal-Wallis H test. P-Value: The probability of observing the effect assuming the null hypothesis is true. CI Lower Bound: Lower bound of the 95% Confidence Interval for the Mean Domains. CI Upper Bound: Upper bound of the 95% Confidence Interval for the Mean Domains. Bitext: The process of aligning bilingual texts by sentence or paragraph. Classify: The task of categorizing data into pre-defined classes. Cluster: The task of grouping a set of objects in such a way that objects in the same group are more similar to each other. InstructRetrieve: Retrieving specific instructions from a corpus. Multilabel Classify: Classification task where each instance may belong to multiple categories. Pair Classify: Classification of pairs of words or entities into predefined categories. Rerank: The process of reordering items to better meet certain criteria. Retrieve: The task of finding relevant information from a large repository. Semantic Similarity: Determining how similar two pieces of text are semantically. Significance: ns $p >= 0.01$, * $p < 0.01$, ** $p < 0.001$, *** $p < 0.0001$.

Created by data-to-paper (AI)

ity" as areas exhibiting significant domain diversity.

Despite these insights, our study does not come without limitations. The scope of the dataset limits the generalizability of our findings across multilingual environments beyond those captured in "mult_tasks.csv". Additionally, our study's focus on specific tasks and resource types could overlook other pertinent factors influencing domain diversity, such as linguistic complexity or cultural variables, which were not accounted for explicitly in our analysis. Further research is needed to understand how these omitted factors might affect resource allocation strategies.

In conclusion, our findings underscore a substantial association between resource types and domain diversity in multilingual tasks. The evidence suggests that certain resource types are more adept at addressing diverse domain applications, with potential implications for improving the efficiency and adaptability of multilingual processing systems. Pursuing future research avenues that explore the intricate interdependencies between domain diversity, cultural nuances, and linguistic subtleties could significantly enhance our understanding and optimization of such multilingual frameworks.

Methods

Data Source

The primary dataset utilized in this research was obtained from Analemma AI's platform, specifically from the FARS proposal FA0042 file named "mult_tasks.csv". This dataset encapsulates various multilingual tasks, where each row corresponds to a distinct resource type and is characterized by several attributes. Key columns include the 'Type' column representing the classification of resources and the 'Domains' column which lists the domains within which each resource is applied. The dataset serves as a foundation for analyzing the relationship between resource types and the diversity of their applications across different domains.

Data Preprocessing

Initial data preprocessing involved handling missing values by excluding rows lacking domain information to ensure the robustness of the subsequent analysis. The 'Domains' data was provided as strings and was systematically converted to list structures. This conversion enabled the determination of domain diversity by calculating the number of unique elements within each list. Consequently, a new column indicating the number of distinct domains

Created by data-to-paper (AI)

associated with each resource was created, facilitating a cleaner and more organized approach to measure domain diversity inherent to each resource type.

Data Analysis

The analytical framework employed a comparative statistical methodology to uncover variations in domain diversity across different resource types. Specifically, a non-parametric statistical test was implemented to evaluate the hypothesis concerning the association between resource types and the diversity of their domain applications. Resource types were grouped based on their classifications, and the number of unique domains for each type was aggregated. This grouping enabled the examination of the median domain numbers across types. Following this, the Kruskal-Wallis test, a robust method suitable for non-normally distributed data, was applied to these groups to determine whether statistically significant differences existed in domain diversity. The statistical findings were contextualized by calculating and comparing the mean number of distinct domains for each resource type, accompanied by standard deviations. To bolster the interpretation of the results, confidence intervals were computed for the mean estimates. These analyses provide critical insights into the heterogeneity of domain coverage across resource types, highlighting potential factors driving resource versatility in multilingual settings.

Code Availability

Custom code used to perform the data preprocessing and analysis, as well as the raw code outputs, are provided in Supplementary Methods.

References

[1] *Designing Courses and Tasks in a Multilingual Perspective.* Channel View Publications, 2019.

[2] Natalia Vanetik and Marina Litvak. *Multilingual Text Analysis: History, Tasks, and Challenges.* WORLD SCIENTIFIC, 2019.

[3] Jannik Strotgen and Michael Gertz. Multilingual and cross-domain temporal tagging. *Language Resources and Evaluation*, 47(2), 2012.

[4] *ANOVA and Kruskal-Wallis Test.* SAGE Publications, Inc, 2017.

Created by data-to-paper (AI)

[5] Priyambada Ambastha and Maunendra Sankar Desarkar. mtransdial: Multilingual dataset for transport domain dialog systems (poster). 2021.

[6] Chuyang Ye, Dongyan Wei, Zhendong Liu, Yuanyi Pang, Yixi Lin, Qinting Jiang, Jingyan Jiang, and Dongbiao He. Datta: Domain diversity aware test-time adaptation for dynamic domain shift data streams. 2025.

[7] *Kruskal-Wallis Test*. Springer Netherlands, 2008.

[8] Daniel Ferres and Horacio Rodriguez. Experiments adapting an open-domain question answering system to the geographical domain using scope-based resources. 2006.

Created by data-to-paper (AI)

A Data Description

Here is the data description, as provided by the user:

```
## General Description
Dataset for FARS proposal FA0042 from https://analemma.ai/fars/.
    File: mult_tasks.csv

## Data Files
The dataset consists of 1 data file:

### "mult_tasks.csv"
Data file for FA0042. File: mult_tasks.csv. Explore columns to
    find statistically testable hypotheses.
```

B Data Exploration

B.1 Code

The Data Exploration was carried out using the following custom code:

```
import pandas as pd

# Load the dataset
df = pd.read_csv('mult_tasks.csv')

# Filename to save the data exploration summary
output_filename = "data_exploration.txt"

# Header write function
def write_section(header, content):
    with open(output_filename, 'a') as f:
        f.write(f"# {header}\n")
        f.write(f"{content}\n\n")

# Data Size
data_size = f"Number of Rows: {df.shape[0]}\nNumber of Columns: {
    ↪ df.shape[1]}"
write_section("Data Size", data_size)

# Summary Statistics (only applicable for numeric columns)
numeric_columns = df.select_dtypes(include=['number']).columns
if not numeric_columns.empty:
    summary_statistics = df.describe().to_string()
else:
    summary_statistics = "Not Applicable"
write_section("Summary Statistics", summary_statistics)
```

Created by data-to-paper (AI)

```
# Categorical Variables
categorical_content = ""
categorical_columns = df.select_dtypes(include=['object', '
    ↪ category']).columns
for col in categorical_columns:
    top_value = df[col].mode()[0]
    categorical_content += f"Column: {col} - Most Common Value: {
        ↪ top_value}\n"
if categorical_content == "":
    categorical_content = "Not Applicable"
write_section("Categorical Variables", categorical_content)

# Missing Values
missing_values = df.isnull().sum()
missing_values_content = missing_values[missing_values > 0].
    ↪ to_string() if missing_values.sum() > 0 else "Not Applicable
    ↪ "
write_section("Missing Values", missing_values_content)

# Other relevant summaries (none identified in the data
    ↪ description)
other_summary = "Not Applicable"
write_section("Other Relevant Summaries", other_summary)
```

B.2 Code Description

The data exploration process is a critical step in understanding the structure, characteristics, and potential issues within a dataset. Here, we provide a high-level explanation of the data exploration code designed to analyze the dataset stored in ‘mult_tasks.csv‘. This exploration is segmented into several key steps, each fulfilling a distinct purpose in the data analysis workflow.

B.3 Data Size

The first step in the exploration process is the determination of the data size. This involves calculating the number of rows and columns present in the dataset. Understanding the dataset size is crucial as it informs whether the dataset is manageable for analysis, or if special computational techniques will be necessary. In the code, this is implemented using the ‘shape‘ attribute of the Pandas DataFrame, which returns the dimensions of the dataset. This basic summary is then written to an output file for record-keeping.

Created by data-to-paper (AI)

B.4 Summary Statistics

For datasets with numeric data, summary statistics provide insights into the central tendency, dispersion, and spread of the data. However, the dataset under consideration primarily contains non-numeric information. Therefore, the code checks for numeric columns, and if such columns are present, it employs the ‘describe‘ method to generate summary statistics like mean, standard deviation, and others. If no numeric columns are present, this step is documented as ”Not Applicable.”

B.5 Categorical Variables

Exploration of categorical variables involves identifying the most common value within each non-numeric column. Categorical data can often reveal patterns or concentrations around certain values, which might be key for hypothesis generation. In the code, this is achieved by selecting object or category data types and determining the mode (most common value) for each column, summarizing the results for documentation.

B.6 Missing Values

The identification of missing values is a crucial step in data preprocessing, as it could affect subsequent analyses and model reliability. The code performs this task by calculating the sum of null entries for each column and only reports columns where missing data is present. This summary of missing values helps inform strategies for data cleaning, such as imputation or exclusion of incomplete cases.

B.7 Other Relevant Summaries

Beyond the standard summaries, the analysis acknowledges the potential need for additional domain-specific insights. However, no other relevant summaries were identified based on the initial data description. This section is thus flagged as ”Not Applicable,” denoting that, under the current scope, no further exploration was necessary.

The results of each of these analyses are consolidated and documented in a text file, ensuring a comprehensive overview of the dataset for further investigation or reporting.

Created by data-to-paper (AI)

B.8 Code Output

data_exploration.txt

```
# Data Size
Number of Rows: 131
Number of Columns: 7

# Summary Statistics
      Unnamed: 0
count        131
mean          65
std        37.96
min            0
25%         32.5
50%           65
75%         97.5
max          130

# Categorical Variables
Column: Name - Most Common Value: AILAStatutes
Column: Type - Most Common Value: Classification
Column: Languages - Most Common Value: {'eng'}
Column: Domains - Most Common Value: ['News', 'Written']
Column: License - Most Common Value: Not specified
Column: Description - Most Common Value: Polish Paraphrase Corpus


# Missing Values
Domains        9
License        9
Description    1

# Other Relevant Summaries
Not Applicable
```

C Data Analysis

C.1 Code

The Data Analysis was carried out using the following custom code:

```python
# IMPORT
from my_utils import df_to_latex, df_to_figure
import pandas as pd
import numpy as np
from scipy.stats import kruskal
import ast
```

Created by data-to-paper (AI)

```python
import pickle

# LOAD DATA
data = pd.read_csv('mult_tasks.csv')

# DATASET PREPARATIONS
# Handle missing values by excluding rows with missing domains
data = data.dropna(subset=['Domains'])

# Convert Domains from string to list and count distinct domains
data['Domains'] = data['Domains'].apply(ast.literal_eval)

# Create a new column for the number of distinct domains
data['Num_Domains'] = data['Domains'].apply(len)

# DESCRIPTIVE STATISTICS
# No descriptive statistics table is needed because the focus is
    ↪ on hypothesis testing for domain diversity.

# PREPROCESSING
# No preprocessing is needed, because categorical description
    ↪ differences are being handled in the analysis phase via
    ↪ existing column information.

# ANALYSIS
## Table df_hypothesis_test:
caption = "Association between Resource Type and Domain Diversity"

# Perform Kruskal-Wallis test
type_groups = data.groupby('Type')['Num_Domains'].apply(list)
kruskal_result = kruskal(*type_groups)

# Create a result dataframe
# Note: Kruskal_Stat and p_value will be constant across all rows
    ↪ because it's one test stat for the entire distribution.
means = [np.mean(group) for group in type_groups]
stds = [np.std(group) for group in type_groups]
df_hypothesis_test = pd.DataFrame({
    'Mean_Num_Domains': means,
    'Std_Num_Domains': stds,
    'Kruskal_Stat': [kruskal_result.statistic] * len(type_groups),
    'p_value': [kruskal_result.pvalue] * len(type_groups)
}, index=type_groups.index)

# Convert to LaTeX
df_to_latex(df_hypothesis_test, 'df_hypothesis_test', caption=
    ↪ caption)

## Figure df_domain_diversity:
```

Created by data-to-paper (AI)

```
caption = "Mean Number of Distinct Domains by Resource Type"

# Calculate confidence intervals
ci_low = df_hypothesis_test['Mean_Num_Domains'] - 1.96 *
    ↪ df_hypothesis_test['Std_Num_Domains'] / np.sqrt([len(group)
    ↪ for group in type_groups])
ci_high = df_hypothesis_test['Mean_Num_Domains'] + 1.96 *
    ↪ df_hypothesis_test['Std_Num_Domains'] / np.sqrt([len(group)
    ↪ for group in type_groups])
df_hypothesis_test['ci_low'] = ci_low
df_hypothesis_test['ci_high'] = ci_high

df_to_figure(df_hypothesis_test, 'df_domain_diversity', caption=
    ↪ caption, kind='bar',
              y=['Mean_Num_Domains'],
              y_ci=[('ci_low', 'ci_high')],
              y_p_value=['p_value'])

# SAVE ADDITIONAL RESULTS
additional_results = {
    'Total number of observations': len(data),
    'Kruskal-Wallis statistic': kruskal_result.statistic,
    'p-value': kruskal_result.pvalue,
}
with open('additional_results.pkl', 'wb') as f:
    pickle.dump(additional_results, f)
```

C.2 Provided Code

The code above is using the following provided functions:

```
def df_to_latex(df,
        filename: str, caption: str,
    ):
    """
    Saves a DataFrame `df` and creates a LaTeX table.
    `filename`, `caption`: as in `df.to_latex`.
    """


def df_to_figure(
        df, filename: str, caption: str,
        x: Optional[str] = None, y: List[str] = None,
        kind: str = 'bar',
        logx: bool = False, logy: bool = False,
        y_ci: Optional[List[str]] = None,
        y_p_value: Optional[List[str]] = None,
    ):
```

Created by data-to-paper (AI)

```
"""
Save a DataFrame `df` and create a LaTeX figure.
Parameters, for LaTex embedding of the figure:
`filename`: Filename for the figure.
`caption`: Caption for the figure.

Parameters for df.plot():
`x`: Column name for x-axis (index by default).
`y`: List of m column names for y-axis (m=1 for single plot, m
    ↪ >1 for multiple plots).
`kind`: only bar is allowed.
`logx` / `logy` (bool): log scale for x/y axis.

`y_ci`: Confidence intervals for errorbars.
    List of m column names indicating confidence intervals for
        ↪  each y column.
    Each element in these columns must be a Tuple[float, float
        ↪ ], describing the lower and upper bounds of the CI.

 `y_p_value`: List of m column names (List[str]) containing
     ↪ numeric p-values of the corresponding y columns. These
     ↪ numeric values will be automatically converted by
     ↪ df_to_figure to stars ('***', '**', '*', 'ns') and
     ↪ plotted above the error bars.

If provided, the length of `y_ci`, and `y_p_value` should be
    ↪ the same as of `y`.

Example:
Suppose, we have:

df_lin_reg_longevity = pd.DataFrame({
    'adjusted_coef': [0.4, ...], 'adjusted_coef_ci': [(0.35,
        ↪ 0.47), ...], 'adjusted_coef_pval': [0.012, ...],
    'unadjusted_coef': [0.2, ...], 'unadjusted_coef_ci':
        ↪ [(0.16, 0.23), ...], 'unadjusted_coef_pval':
        ↪ [0.0001, ...],
}, index=['var1', ...])

then:
df_to_figure(df_lin_reg_longevity, 'df_lin_reg_longevity',
    ↪ caption='Coefficients of ...', kind='bar',
    y=['adjusted_coef', 'unadjusted_coef'],
    y_ci=['adjusted_coef_ci', 'unadjusted_coef_ci'],
    y_p_value=['adjusted_coef_pval', 'unadjusted_coef_pval'])
"""
```

Created by data-to-paper (AI)

C.3 Code Description

The data analysis process involves several distinct steps aimed at exploring the association between resource types and domain diversity within a multilingual dataset. This procedure can be broken down into data preparation, hypothesis testing, and visualization.

C.4 Data Preparation

The initial step involves data cleansing and transformation to ensure that the dataset is suitable for analysis. Specifically, rows with missing domain information are removed to maintain the integrity of the dataset. The 'Domains' column, initially a string representation, is converted into a list of domains per resource. Subsequently, a new column, 'Num_Domains', is created to represent the count of distinct domains associated with each resource, facilitating subsequent statistical analysis.

C.5 Analysis

The primary aim of the analysis is to test for differences in domain diversity across different resource types. For this purpose, the Kruskal-Wallis test, a non-parametric method for comparing more than two independent groups, is employed. The dataset is grouped by the 'Type' of resource, and the number of distinct domains (*Num_Domains*) serves as the metric for comparison. By evaluating differences in median domain counts across resource types, the test statistically determines if at least one group differs significantly from the others.

C.6 Hypothesis Testing and Descriptive Tables

Results from the Kruskal-Wallis test are summarized in a result dataframe, denoted as `df_hypothesis_test`. This table includes the mean and standard deviation of the number of distinct domains for each resource type, alongside the Kruskal-Wallis statistic and the associated p-value, providing a comprehensive view of the findings. The table is formatted and exported using LaTeX to streamline inclusion in publication materials.

C.7 Visualization

To visually represent the data, a bar chart is generated, illustrating the mean number of distinct domains for each type of resource, accompanied by

confidence intervals. These intervals provide insight into the reliability of the mean estimates across groups, while annotations of p-values on the plot communicate the statistical significance of observed differences.

C.8 Storage of Additional Results

Supplementary analysis results, including the total number of observations, the Kruskal-Wallis statistic, and the associated p-value, are serialized and stored in a pickle file for potential future use or reference, ensuring reproducibility and transparency of the analysis.

This methodological framework facilitates the exploration of relationships within the dataset, enabling data-driven insights into domain diversity across different resource types within a multilingual context.

C.9 Code Output

df_domain_diversity.pkl

```
                         Mean_Num_Domains Std_Num_Domains
                             Kruskal_Stat p_value ci_low ci_high
Type
BitextMining                          2.615           1.496
    18.46   0.018  1.802   3.428
Classification                        2.256          0.9424
    18.46   0.018  1.974   2.537
Clustering                            2.188          0.3903
    18.46   0.018  1.996   2.379
InstructionRetrieval                      2               0
    18.46   0.018      2       2
MultilabelClassification                2.6             0.8
    18.46   0.018  1.899   3.301
PairClassification                      3.3           1.487
    18.46   0.018  2.379   4.221
Reranking                               2.6          0.4899
    18.46   0.018  2.171   3.029
Retrieval                               2.2             0.4
    18.46   0.018  1.998   2.402
STS                                   3.167           1.518
    18.46   0.018  2.308   4.026
```

df_hypothesis_test.pkl

```
                         Mean_Num_Domains Std_Num_Domains
                             Kruskal_Stat p_value
Type
```


Created by data-to-paper (AI)

Created by data-to-paper (AI)

```
BitextMining                          2.615           1.496
    18.46   0.018
Classification                        2.256          0.9424
    18.46   0.018
Clustering                            2.188          0.3903
    18.46   0.018
InstructionRetrieval                      2               0
    18.46   0.018
MultilabelClassification                2.6             0.8
    18.46   0.018
PairClassification                      3.3           1.487
    18.46   0.018
Reranking                               2.6          0.4899
    18.46   0.018
Retrieval                               2.2             0.4
    18.46   0.018
STS                                   3.167           1.518
    18.46   0.018
```

additional_results.pkl

```
{
    'Total number of observations': 122,
    'Kruskal-Wallis statistic': 18.46,
    'p-value': 0.018,
}
```

D LaTeX Table Design

D.1 Code

The LaTeX Table Design was carried out using the following custom code:

```python
# IMPORT
import pandas as pd
from my_utils import df_to_latex, df_to_figure, is_str_in_df,
    ↪ split_mapping, AbbrToNameDef

# PREPARATION FOR ALL TABLES AND FIGURES
# Define mapping for all df labels that need to be renamed and/or
    ↪ glossary defined.
all_mapping: AbbrToNameDef = {
    # Provide scientific names and glossary definitions for
        ↪ columns:
    'Mean_Num_Domains': ('Mean Domains', 'Average number of
        ↪ distinct domains associated with each Type'),
    'Std_Num_Domains': ('SD Domains', 'Standard deviation of
        ↪ distinct domains associated with each Type'),
```

Created by data-to-paper (AI)

```
    'Kruskal_Stat': ('Kruskal-Wallis Statistic', 'Test statistic
        ↪ from Kruskal-Wallis H test'),
    'p_value': ('P-Value', 'The probability of observing the
        ↪ effect assuming the null hypothesis is true'),
    'ci_low': ('CI Lower Bound', 'Lower bound of the 95%
        ↪ Confidence Interval for the Mean Domains'),
    'ci_high': ('CI Upper Bound', 'Upper bound of the 95%
        ↪ Confidence Interval for the Mean Domains'),
    # Provide full definitions for index labels:
    'BitextMining': ('Bitext', 'The process of aligning bilingual
        ↪ texts by sentence or paragraph'),
    'Classification': ('Classify', 'The task of categorizing data
        ↪ into pre-defined classes'),
    'Clustering': ('Cluster', 'The task of grouping a set of
        ↪ objects in such a way that objects in the same group are
        ↪  more similar to each other'),
    'InstructionRetrieval': ('InstructRetrieve', 'Retrieving
        ↪ specific instructions from a corpus'),
    'MultilabelClassification': ('Multilabel Classify', '
        ↪ Classification task where each instance may belong to
        ↪ multiple categories'),
    'PairClassification': ('Pair Classify', 'Classification of
        ↪ pairs of words or entities into predefined categories'),
    'Reranking': ('Rerank', 'The process of reordering items to
        ↪ better meet certain criteria'),
    'Retrieval': ('Retrieve', 'The task of finding relevant
        ↪ information from a large repository'),
    'STS': ('Semantic Similarity', 'Determining how similar two
        ↪ pieces of text are semantically'),
}

# Process df_domain_diversity
df_domain_diversity = pd.read_pickle('df_domain_diversity.pkl')

# Not Applicable: # Format values.

# Rename rows and columns
mapping = dict((k, v) for k, v in all_mapping.items() if
    ↪ is_str_in_df(df_domain_diversity, k))
abbrs_to_names, glossary = split_mapping(mapping)
df_domain_diversity.rename(columns=abbrs_to_names, index=
    ↪ abbrs_to_names, inplace=True)

# Choose to present as a figure
df_to_figure(
    df_domain_diversity, 'df_domain_diversity_formatted',
    caption="Mean Number of Distinct Domains by Resource Type with
        ↪  Confidence Intervals",
    glossary=glossary,
```

Created by data-to-paper (AI)

```
    kind='bar',
    y=['Mean Domains'],
    y_ci=[('CI Lower Bound', 'CI Upper Bound')],
    y_p_value=['P-Value']
)

# Process df_hypothesis_test
df_hypothesis_test = pd.read_pickle('df_hypothesis_test.pkl')

# Not Applicable: # Format values.

# Rename rows and columns
mapping = dict((k, v) for k, v in all_mapping.items() if
    ↪ is_str_in_df(df_hypothesis_test, k))
abbrs_to_names, glossary = split_mapping(mapping)
df_hypothesis_test.rename(columns=abbrs_to_names, index=
    ↪ abbrs_to_names, inplace=True)

# Handle constant column
kruskal_statistic_unique = df_hypothesis_test['Kruskal-Wallis
    ↪ Statistic'].unique()
assert len(kruskal_statistic_unique) == 1
df_hypothesis_test.drop(columns=['Kruskal-Wallis Statistic'],
    ↪ inplace=True)

# Filter glossary to match the existing df columns and index
glossary = {k: v for k, v in glossary.items() if k in
    ↪ df_hypothesis_test.columns or k in df_hypothesis_test.index}

# Choose to present as a table
df_to_latex(
    df_hypothesis_test, 'df_hypothesis_test_formatted',
    caption="Association between Resource Type and Domain
        ↪ Diversity",
    note=f"For all rows, the Kruskal-Wallis Statistic is {
        ↪ kruskal_statistic_unique[0]}. The P-Value indicates
        ↪ significance as reported.",
    glossary=glossary
)
```

D.2 Provided Code

The code above is using the following provided functions:

```
def df_to_latex(df,
        filename: str, caption: str,
        note: str = None,
        glossary: Dict[Any, str] = None,
```

Created by data-to-paper (AI)

```
    ):
    """
    Saves a DataFrame `df` and creates a LaTeX table.
    `filename`, `caption`: as in `df.to_latex`.
    `note`: Note to be added below the table caption.
    `glossary`: Glossary for the table.
    """


def df_to_figure(
        df, filename: str, caption: str,
        note: str = None, glossary: Dict[Any, str] = None,
        x: Optional[str] = None, y: List[str] = None,
        kind: str = 'bar',
        logx: bool = False, logy: bool = False,
        y_ci: Optional[List[str]] = None,
        y_p_value: Optional[List[str]] = None,
        xlabel: str = None, ylabel: str = None,
    ):
    """
    Save a DataFrame `df` and create a LaTeX figure.
    Parameters, for LaTex embedding of the figure:
    `filename`: Filename for the figure.
    `caption`: Caption for the figure.
    `note`: Note to be added below the figure caption.
    `glossary`: Glossary for the figure.

    Parameters for df.plot():
    `x`: Column name for x-axis (index by default).
    `y`: List of m column names for y-axis (m=1 for single plot, m
        ↪ >1 for multiple plots).
    `kind`: only bar is allowed.
    `logx` / `logy` (bool): log scale for x/y axis.
    `xlabel`: Label for the x-axis.
    `ylabel`: Label for the y-axis.

    `y_ci`: Confidence intervals for errorbars.
        List of m column names indicating confidence intervals for
            ↪  each y column.
        Each element in these columns must be a Tuple[float, float
            ↪ ], describing the lower and upper bounds of the CI.

     `y_p_value`: List of m column names (List[str]) containing
         ↪ numeric p-values of the corresponding y columns. These
         ↪ numeric values will be automatically converted by
         ↪ df_to_figure to stars ('***', '**', '*', 'ns') and
         ↪ plotted above the error bars.
```

Created by data-to-paper (AI)

```
    If provided, the length of `y_ci`, and `y_p_value` should be
        ↪ the same as of `y`.

    Example:
    Suppose, we have:

    df_lin_reg_longevity = pd.DataFrame({
        'adjusted_coef': [0.4, ...], 'adjusted_coef_ci': [(0.35,
            ↪ 0.47), ...], 'adjusted_coef_pval': [0.012, ...],
        'unadjusted_coef': [0.2, ...], 'unadjusted_coef_ci':
            ↪ [(0.16, 0.23), ...], 'unadjusted_coef_pval':
            ↪ [0.0001, ...],
    }, index=['var1', ...])

    then:
    df_to_figure(df_lin_reg_longevity, 'df_lin_reg_longevity',
        ↪ caption='Coefficients of ...', kind='bar',
        y=['adjusted_coef', 'unadjusted_coef'],
        y_ci=['adjusted_coef_ci', 'unadjusted_coef_ci'],
        y_p_value=['adjusted_coef_pval', 'unadjusted_coef_pval'])
    """


def is_str_in_df(df: pd.DataFrame, s: str):
    return any(s in level for level in getattr(df.index, 'levels',
        ↪  [df.index]) + getattr(df.columns, 'levels', [df.columns
        ↪ ]))

AbbrToNameDef = Dict[Any, Tuple[Optional[str], Optional[str]]]

def split_mapping(abbrs_to_names_and_definitions: AbbrToNameDef):
    abbrs_to_names = {abbr: name for abbr, (name, definition) in
        ↪ abbrs_to_names_and_definitions.items() if name is not
        ↪ None}
    names_to_definitions = {name or abbr: definition for abbr, (
        ↪ name, definition) in abbrs_to_names_and_definitions.
        ↪ items() if definition is not None}
    return abbrs_to_names, names_to_definitions
```

D.3 Code Output

df_domain_diversity_formatted.pkl

```
\begin{figure}[htbp]
\centering
\includegraphics{df_domain_diversity_formatted.png}
\caption{Mean Number of Distinct Domains by Resource Type with
    Confidence Intervals
```

Created by data-to-paper (AI)

```
Mean Domains: Average number of distinct domains associated with
    each Type.
SD Domains: Standard deviation of distinct domains associated with
     each Type.
Kruskal-Wallis Statistic: Test statistic from Kruskal-Wallis H
    test.
P-Value: The probability of observing the effect assuming the null
     hypothesis is true.
CI Lower Bound: Lower bound of the 95\% Confidence Interval for
    the Mean Domains.
CI Upper Bound: Upper bound of the 95\% Confidence Interval for
    the Mean Domains.
Bitext: The process of aligning bilingual texts by sentence or
    paragraph.
Classify: The task of categorizing data into pre-defined classes.
Cluster: The task of grouping a set of objects in such a way that
    objects in the same group are more similar to each other.
InstructRetrieve: Retrieving specific instructions from a corpus.
Multilabel Classify: Classification task where each instance may
    belong to multiple categories.
Pair Classify: Classification of pairs of words or entities into
    predefined categories.
Rerank: The process of reordering items to better meet certain
    criteria.
Retrieve: The task of finding relevant information from a large
    repository.
Semantic Similarity: Determining how similar two pieces of text
    are semantically.
Significance: ns p $>$= 0.01, * p $<$ 0.01, ** p $<$ 0.001, *** p
    $<$ 0.0001.}
\label{figure:df-domain-diversity-formatted}
\end{figure}
% This latex figure presents "df_domain_diversity_formatted.png",
% which was created from the df:
%
% index,"Mean Domains","SD Domains","Kruskal-Wallis Statistic","P-
    Value","CI Lower Bound","CI Upper Bound"
% "Bitext",2.615,1.496,18.46,0.018,1.802,3.428
% "Classify",2.256,0.9424,18.46,0.018,1.974,2.537
% "Cluster",2.188,0.3903,18.46,0.018,1.996,2.379
% "InstructRetrieve",2,0,18.46,0.018,2,2
% "Multilabel Classify",2.6,0.8,18.46,0.018,1.899,3.301
% "Pair Classify",3.3,1.487,18.46,0.018,2.379,4.221
% "Rerank",2.6,0.4899,18.46,0.018,2.171,3.029
% "Retrieve",2.2,0.4,18.46,0.018,1.998,2.402
% "Semantic Similarity",3.167,1.518,18.46,0.018,2.308,4.026
%
% To create the figure, this df was plotted with the command:
```

Created by data-to-paper (AI)

```
%
% df.plot(kind='bar', y=['Mean Domains'])
%
% Confidence intervals for y-values were then plotted based on
    column: [('CI Lower Bound', 'CI Upper Bound')].
%
% P-values for y-values were taken from column: ['P-Value'].
%
% These p-values were presented above the data points as stars (
    with significance threshold values indicated in the figure
    caption).
```

df_hypothesis_test_formatted.pkl

```
\begin{table}[h]
\caption{Association between Resource Type and Domain Diversity}
\label{table:df-hypothesis-test-formatted}
\begin{threeparttable}
\renewcommand{\TPTminimum}{\linewidth}
\makebox[\linewidth]{%
\begin{tabular}{lrrl}
\toprule
 & Mean Domains & SD Domains & P-Value \\
Type &  &  &  \\
\midrule
\textbf{Bitext} & 2.615 & 1.496 & 0.018 \\
\textbf{Classify} & 2.256 & 0.9424 & 0.018 \\
\textbf{Cluster} & 2.188 & 0.3903 & 0.018 \\
\textbf{InstructRetrieve} & 2 & 0 & 0.018 \\
\textbf{Multilabel Classify} & 2.6 & 0.8 & 0.018 \\
\textbf{Pair Classify} & 3.3 & 1.487 & 0.018 \\
\textbf{Rerank} & 2.6 & 0.4899 & 0.018 \\
\textbf{Retrieve} & 2.2 & 0.4 & 0.018 \\
\textbf{Semantic Similarity} & 3.167 & 1.518 & 0.018 \\
\bottomrule
\end{tabular}}
\begin{tablenotes}
\footnotesize
\item For all rows, the Kruskal-Wallis Statistic is
    18.460439621974576. The P-Value indicates significance as
    reported.
\item \textbf{Mean Domains}: Average number of distinct domains
    associated with each Type
\item \textbf{SD Domains}: Standard deviation of distinct domains
    associated with each Type
\item \textbf{P-Value}: The probability of observing the effect
    assuming the null hypothesis is true
\item \textbf{Bitext}: The process of aligning bilingual texts by
    sentence or paragraph
```

```
\item \textbf{Classify}: The task of categorizing data into pre-
    defined classes
\item \textbf{Cluster}: The task of grouping a set of objects in
    such a way that objects in the same group are more similar to
    each other
\item \textbf{InstructRetrieve}: Retrieving specific instructions
    from a corpus
\item \textbf{Multilabel Classify}: Classification task where each
     instance may belong to multiple categories
\item \textbf{Pair Classify}: Classification of pairs of words or
    entities into predefined categories
\item \textbf{Rerank}: The process of reordering items to better
    meet certain criteria
\item \textbf{Retrieve}: The task of finding relevant information
    from a large repository
\item \textbf{Semantic Similarity}: Determining how similar two
    pieces of text are semantically
\end{tablenotes}
\end{threeparttable}
\end{table}
```

Created by data-to-paper (AI)